

Rapport de Projet de Microcontrôleurs

05/2025

Groupe 30

El Mouhtar William (373549)

Abela Oliver (372178)

Titre : Poste de Pilotage

Remerciement :

Nous tenons à exprimer notre profonde gratitude au Prof. Alexandre Schmid pour avoir initié ce projet. Nous remercions également les assistants pour leur aide précieuse, qui a grandement contribué à la progression du travail. Cette expérience a été l'occasion de consolider nos compétences en langage assembleur, tout en développant notre capacité à collaborer efficacement en binôme.

Par ailleurs, nous avons particulièrement apprécié la liberté qui nous a été accordée dans la conduite de ce projet, une autonomie qui a favorisé l'apprentissage et la créativité.

Description générale de l'application :

Dans le cadre du projet de printemps 2025, nous avons utilisé le kit STK-300 pour développer une interface de type *poste de pilotage* pour un bateau. La navigation dans les différents menus se fait à l'aide d'une télécommande infrarouge au format RC5 : le bouton 1 permet d'afficher la température, le bouton 'arrière' lance la mesure de distance via un capteur SHARP, et un bouton poussoir déclenche un signal d'alerte SOS.

Lorsque l'utilisateur presse le **bouton 1**, la température ambiante s'affiche sur le LCD. Un code couleur via les LEDs permet une lecture rapide : **bleu** s'il fait frais (moins de 26 °C), **rouge** s'il fait chaud (26 °C ou plus). Ainsi, d'un seul coup d'œil, le marin peut juger des conditions extérieures.

Le **bouton 'arrière'** active la mesure de distance via le capteur SHARP. La valeur détectée est affichée sur le LCD. Si celle-ci est supérieure ou égale à 200 (ce qui correspond à une distance inférieure ou égale à 30 cm), les LEDs s'allument en **rouge** et le message "**STOP !**" est affiché en complément, avertissant le marin d'une proximité critique avec un obstacle. À l'inverse, pour une valeur inférieure à 200 (distance > 30 cm), les LEDs deviennent **vertes**, signalant que l'espace est suffisant pour manœuvrer ou s'amarrer.

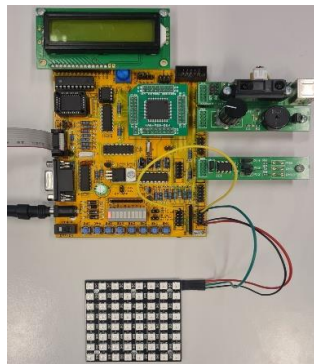
Enfin, en appuyant sur le **bouton poussoir 0**, un message "**SOS**" s'affiche immédiatement à l'écran, accompagné d'une simulation d'un signal SOS en **orange**. Il permet au marin d'alerter rapidement les personnes présentes au port ou à proximité afin qu'elles puissent intervenir et lui porter assistance dans les plus brefs délais.

Mode d'emploi :

1.Mise en opération du système :

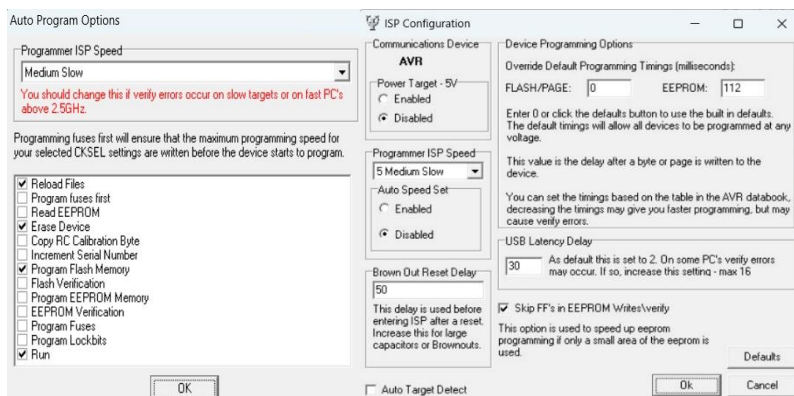
-Commencez par connecter la carte STK-300 équipée du microcontrôleur ATmega128 à l'ordinateur. Branchez le programmeur ISP à un port USB de l'ordinateur. À l'aide d'un câble plat gris, reliez le programmeur à la carte STK-300. Connectez le bloc d'alimentation 9V au secteur 220V, puis mettez la carte sous tension à l'aide du petit interrupteur noir situé en bas à gauche de la carte. Une fois l'alimentation activée, la LED "**ON**" doit s'allumer, confirmant que la carte est bien alimentée.

-Ensuite, branchez le périphérique M2 sur le Port E, le périphérique M4 sur le Port F, le périphérique M5 sur le Port B, l'écran LCD en haut a gauche de la carte et enfin le WS2812b au port D de la manière suivante : du côté des LEDs, câble rouge sur V+, noir sur V-, vert sur IN. Du côté de la carte : noir sur GND, vert sur le pin1, rouge sur le pin 7. Et finalement branchez le PIN0 du port D au PIN0 des boutons poussoirs.



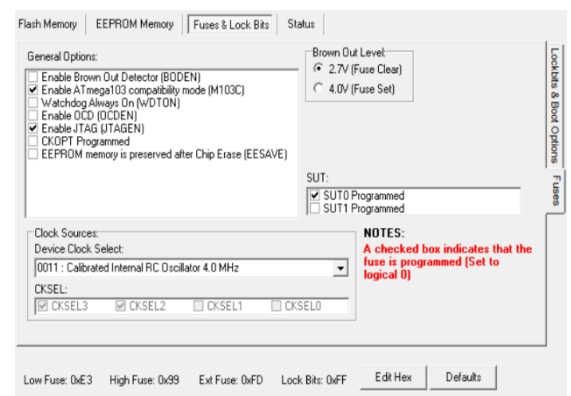
a : Carte STK 300 avec périphériques placés correctement

- Ouvrir le logiciel Atmel Studio 7.0 et importer le code source. - Depuis Atmel Studio 7.0, si le plugin a été installé, cliquer sur la commande Tools→AVRISP-U. AVRISP-U connaît automatiquement le chemin vers le fichier « .hex ». Faire ensuite ces quelques configurations dans AVRISP-U :



b: Autoprogram options

c: Bouton Setup



d: onglet Fuses & Lock bits

2. Utilisation par l'utilisateur :

Lorsque le système est en marche, il détecte les signaux infrarouges via le capteur IR intégré. L'utilisateur peut alors envoyer des commandes à l'aide de la télécommande, que le système interprète et exécute.

Fonctionnalités disponibles :

- **Affichage de la température**
En appuyant sur le bouton **1**, la température ambiante est mesurée et affichée sur le LCD.
→ Si elle est **inférieure à 26 °C**, les LEDs s'allument en **bleu**.
→ Si elle est **≥ 26 °C**, les LEDs passent au **rouge**.
- **Mesure de distance**
En appuyant sur le bouton '**arrière**', le système affiche la distance mesurée par le capteur SHARP.
→ Si la valeur est **≥ 200** (≤ 30 cm), les LEDs deviennent **rouges** et "**STOP !**" est affiché.
→ Si la valeur est **< 200**, les LEDs passent au **vert**.
- **Signal SOS**
À tout moment, l'utilisateur peut presser le **bouton poussoir 0** de la carte STK-300. Cela interrompt l'exécution en cours, affiche "**SOS**" sur le LCD et fait **un signal d'alerte en orange**. Une fois l'alerte lancée, une nouvelle commande peut être choisie via la télécommande.

Toutes les informations sont affichées en temps réel sur l'écran LCD connecté au microcontrôleur.

Description technique et Fonctionnement du Programme :

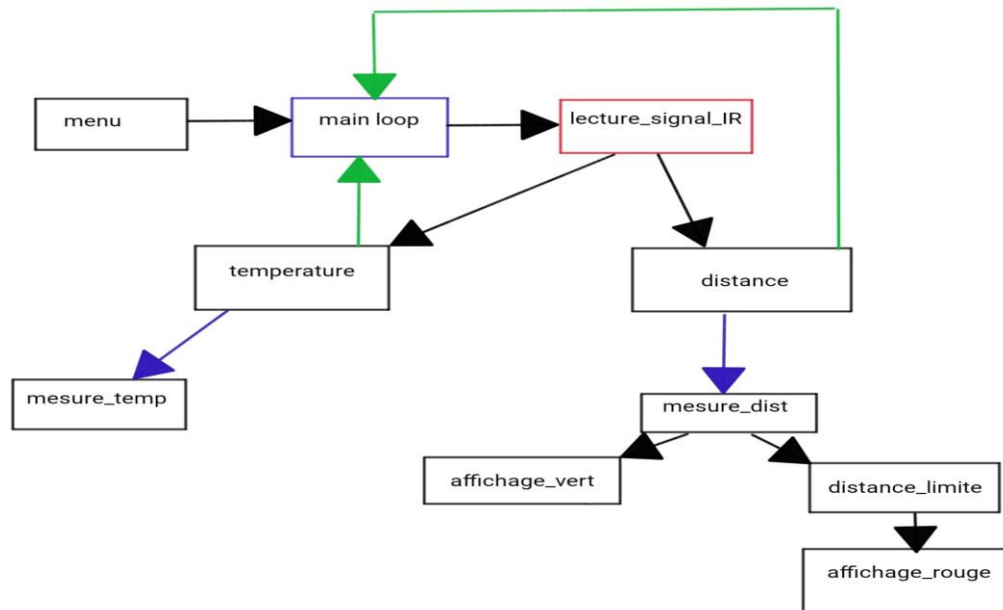
L'application est développée en **assembleur** pour programmer le microcontrôleur **ATmega128** de la carte **STK-300**. Elle repose sur plusieurs fichiers d'inclusion définissant macros, registres, constantes et sous-routines, ce qui rend le code plus clair et modulaire.

Configuration des ports:

- **Port D** (entrée) : détecte l'appui sur le **bouton poussoir 0**.
- **Port B** (sortie) : contrôle le **capteur de température**.
- **Port E** : pilote le **buzzer** et le **capteur infrarouge**.
- **ext_int0** : broche d'**interruption externe** déclenchant une action dès qu'un signal est reçu.

Le programme suit une **approche top-down** : il est conçu globalement, puis décomposé en modules détaillés, ce qui facilite la compréhension des interactions entre les différentes parties du système.

Modules constituant le programme :



e : Illustration du fonctionnement général du projet

1. Interruption ext_int0

Déclenchée par la broche INT0, cette routine s'active immédiatement lors d'un signal externe (ex. appui sur le bouton SOS). Elle exécute les sous-routines qui affichent "SOS" sur le LCD et activent le clignotement d'alerte des LEDs.

2. Routine de réinitialisation (reset)

Lancée à l'allumage ou après une réinitialisation, elle initialise les éléments essentiels du système :

- Chargement du pointeur de pile (LDSP)
- Configuration des ports I/O (OUTI)
- Initialisation de l'interface ADC pour les signaux analogiques
- Paramétrage des interruptions externes (EIMSK, EICRA)
- Activation globale des interruptions (sei)

Cette routine constitue la base du fonctionnement du programme.

3. Routine principale (main)

Au cœur du programme, cette boucle infinie (main_loop) traite les commandes reçues via la télécommande IR.

- lecture_signal_IR : Détecte et lit le signal IR, stocke la valeur dans b0 pour déterminer l'action à exécuter.

- **temperature** : Si la commande demande l'affichage de la température, le programme appelle `mesure_temp` (depuis `temperature.asm`). La température est affichée, et les LEDs s'illuminent en rouge ou bleu selon la valeur mesurée. Retour à la boucle.

- **distance** : Si la commande concerne la distance, le programme appelle `mesure_dist` (depuis `distance.asm`) et affiche la distance mesurée par le capteur SHARP :

→ ≥ 200 (≤ 30 cm) : LEDs rouges + message "STOP !"

→ < 200 : LEDs vertes

Une fois l'action terminée, retour à la boucle pour attendre une nouvelle commande.

Présentation des fichiers d'inclusion :

1 .include "macros.asm"

Contient des macros qui simplifient l'écriture du code en remplaçant des séquences répétitives par une instruction unique. Cela améliore la lisibilité et réduit les erreurs.

2 .include "definitions.asm"

Fichier centralisant toutes les définitions de registres, constantes et adresses mémoire utilisées dans le programme. Il facilite la maintenance et les modifications globales.

3 .include "lcd.asm"

Regroupe les sous-routines nécessaires à l'utilisation de l'écran LCD : initialisation, positionnement du curseur, effacement et affichage de texte.

4 .include "printf.asm"

Permet un affichage formaté sur le LCD, avec des fonctions proches de la commande `printf` du langage C. Idéal pour afficher des données de manière claire.

5 .include "ir_rc5.asm"

Gère la réception et le décodage des signaux infrarouges envoyés par une télécommande au format RC5. Il interprète les commandes utilisateur pour exécuter les actions correspondantes.

6 .include "distance.asm"

Contient les fonctions liées à la mesure de distance via le capteur SHARP. Il lance la conversion analogique-numérique et traite les valeurs pour déterminer l'affichage et l'état des LEDs.

7 .include "wire1.asm"

Fournit les routines de communication via l'interface 1-Wire®, utilisée par certains capteurs numériques. Il prend en charge l'initialisation, l'envoi et la réception de données.

8 .include "temperature.asm"

Basé sur `wire1.asm`, ce fichier est dédié à la lecture de la température à partir d'un capteur compatible 1-Wire®. Il contient les routines spécifiques de mesure et d'affichage.

9 .include "couleur.asm"

Gère l'activation des LEDs en fonction de l'état du système. Il permet de modifier dynamiquement la couleur des LEDs (rouge, vert, bleu, orange) pour informer l'utilisateur de la situation.

Description de détail de l'accès des périphériques :

→ LCD :

- Comme définit dans lcd.asm : les deux adresses définies LCD_IR et LCD_DR à 0x8000 et 0xC000 respectivement permettent de communiquer avec l'écran LCD en écrivant les valeurs dans ces registres.
- La sous-routine LCD_init permet l'initialisation de l'afficheur LCD pour qu'il soit prêt à recevoir des caractères ASCII, positionner correctement le curseur, effacer l'écran... Cette sous-routine a été utilisée dans le main du menu_main.asm et donc appelée au démarrage.
- LCD_home et LCD_clear sont utilisés respectivement pour remettre le curseur à sa position d'origine et pour effacer le contenu affiché par la LCD.
- Le réglage de la position du curseur sur la LCD est possible grâce à la sous-routine LCD_pos. Ce dernier a été utilisé pour le passage à la ligne suivante en faisant appel à la sous-routine LCD_lf. L'adresse est calculée en fonction du positionnement donnée en argument (ici a0 = \$40 donc le début de la ligne 2)

→ Capteur de température :

- temperature.asm utilise le protocole 1-wire pour communiquer avec le capteur de température.
- La commande « convertT » a pour but d'initier la conversion de la température pour ensuite être lue par le capteur avec la commande « readScratchpad ». Ensuite, pour lire et écrire les données lues par le capteur, l'utilisation des routines « wire1_write » et « wire1_read » ont été nécessaires.
- L'affichage de la température est fait ensuite avec le PRINTF LCD qui permet

→ Capteur de Distance :

- Le registre ADCSR est configuré de manière à activer le convertisseur analogique-numérique (ADC), avec un pré diviseur de fréquence fixé à 64, garantissant une fréquence d'échantillonnage adaptée.
- Le capteur de distance est connecté à une broche analogique du microcontrôleur. Pour sélectionner cette entrée, le programme configure le registre ADMUX avec la valeur **3**, ce qui correspond au canal du potentiomètre (POT). Cette opération permet d'acheminer la tension du capteur vers le convertisseur ADC.
- Le programme lance une conversion ADC en positionnant le bit ADSC dans le registre ADCSR. Il attend ensuite jusqu'à ce que le bit ADIF s'active, signalant que la conversion est terminée. Le résultat de la conversion est ensuite récupéré : le LSByte est lu dans le registre ADCL et le MSByte dans ADCH.

- Le programme efface ensuite l'écran LCD à l'aide de la commande `lcd_clear`. Il utilise la fonction `PRINTF` pour afficher les résultats sur l'écran.

→ Télécommande infrarouge (IR) :

-le code pour la sous routine loop dans le `ir_rc5` a été implémentée dans `lecture_signal_IR` qui a permis la lecture de l'information transmise par la télécommande pour ensuite la stocker dans `b0`. Cette valeur sera traitée par des branchement pour gérer différentes fonctionnalités pour différents boutons

→ ws2812b LED :

-les sous-routines affichage_X (X= rouge, bleu, vert, orange, éteint) ont été fait en créent les sous- routines rouge, bleu, vert, orange, éteint respectivement ainsi qu'à l'aide d'une boucle avec 64 répétitions (pour les 64 LEDs) . `petit_triangle` et `grand_triangle` ont été fait avec 64 `rcall` pour la raison de leurs formes.

-`ws2812b`) `_freeze_frame` a été utilisé après chaque affichage_X garder les LED a l'affichage voulu sans que ça s'efface immédiatement après la fin de l'affichage.

Remarques :

-On trouve bien plusieurs branchements dans la `main_loop` du `menu_main.asm` pour utiliser juste 2 boutons. Ceci est dû par un défaut de la télécommande (envoie un signal différent à chaque appuie du même bouton plutôt qu'un seul signal bien défini) la solution a été de **1-** choisir 2 boutons n'ayant aucun code/signal en commun et **2-** trouver les codes qui se répètent pour chaque bouton et les ajouter avec un branchement additionnel. Comme vu dans la vidéo, cette méthode fonctionne sans défaut malgré la méthode de résolution non élégante pour le code. Un fonctionnement correct de la télécommande aurait effectivement entraîné la suffisance de l'utilisation de 2 branchements différents pour la température et la distance

-Pour la première utilisation du mode température, celui-ci affiche +85 degrés, et affiche ensuite la valeur correcte pour le reste des appuis du bouton 1. Ceci est la valeur « Power-on reset » précisée par la documentation du capteur de température - Plus précisément la page 4 du document DS18B20.pdf – comme dans la capture d'écran ci-dessous. Une solution aurait été d'ajouter un `WAIT_MS` d'environ 700 mais ceci brise le principe du fonctionnement en temps réel.

Table 1. Temperature/Data Relationship

TEMPERATURE (°C)	DIGITAL OUTPUT (BINARY)	DIGITAL OUTPUT (HEX)
+125	0000 0111 1101 0000	07D0h
+85*	0000 0101 0101 0000	0550h
+25.0625	0000 0001 1001 0001	0191h
+10.125	0000 0000 1010 0010	00A2h
+0.5	0000 0000 0000 1000	0008h
0	0000 0000 0000 0000	0000h
-0.5	1111 1111 1111 1000	FF8h
-10.125	1111 1111 0101 1110	FF5Eh
-25.0625	1111 1110 0110 1111	FE6Fh
-55	1111 1100 1001 0000	FC90h

*The power-on reset value of the temperature register is +85°C.

Annexes :

Menu_main.asm :

```
/*
 * menu_main.asm
 *
 */

#include "macros.asm"      ; macro definitions
#include "definitions.asm" ; register/constant definitions

.org 0      ; Adresse du reset + interruptions du bouton 0
    jmp reset
    jmp ext_int0

; !! ////////////////////////////////// RESET SOUS-ROUTINE POUR MAIN ////////////////////////////////// !!
reset:
    LDSP      RAMEND      ; load le SP
    OUTI      DDRB,0xff    ; configure le portB en output
    OUTI      DDRD,0x00    ; configure portD to input (bouton poussoir 0)

    ;Pour le detecteur de distance (distance.asm)
    OUTI      ADCSR,(1<<ADEN)+6 ; AD Enable, PS=CK/64
    OUTI      ADMUX, 3      ; select channel POT (potentiometer)

    ; Pour les interruptions
    OUTI      EIMSK,0b00000001 ; enable INT0..INT3
    OUTEI     EICRA, 0b00000010
    sei

    rjmp main

; !! ////////////////////////////////// INTERRUPTION SOUS-ROUTINE POUR BOUTON DU REMOTE ////////////////////////////////// !!
ext_int0:      ; routine d'interruption lorsque le bouton de la telecommande est presse

    rcall print_SOS ; print SOS sur LCD
    rcall petit_triangle
    rcall ws2812b_freeze_frame
    WAIT_MS 100
    rcall grand_triangle
    rcall ws2812b_freeze_frame ;petite animation
    reti

; !! ////////////////////////////////// INCLUDE LES LIBRAIRIES ////////////////////////////////// !!
#include "lcd.asm"      ; lcd sous-routines
#include "printf.asm"   ; printf sous-routines
#include "ir_rc5.asm"   ; Infra Rouge decodeur sous-routines
#include "distance.asm" ; detecteur de distance sous-routines
#include "wire1.asm"    ; Dallas 1-wire(R) routines
#include "temperature.asm" ; module temperature
#include "couleurs.asm" ; sous routines LED ws2812b

; !! ////////////////////////////////// DEFINITIONS ////////////////////////////////// !!
.equ distance_de_transition = 200      ; distance de transition vert/rouge
.equ T50 = 1778      ; bit period T50= 1778 usec
```

```

; !! ////////////////// MAIN ////////////////// !!

// sous-routine read IR signal principale pour le controle du projet
lecture_signal_IR:
    ; recois le signal IR et le stock dans b0
    CLR2    b1,b0        ; clear les 2 registres
    ldi     b2,14        ; load bit-counter
    WP1 PINE,IR          ; Wait si Pin=1
    WAIT_US (T50/4)      ; attend un quart de period

    loop_code: P2C      PINE,IR    ; move Pin to Carry (P2C)
                ROL2      b1,b0
                WAIT_US    (T50-4)    ; compensation
                DJNZ      b2,loop_code
                com      b0          ; complement de b0
    WAIT_US 1000

    ret

// main code du projet
main:

    ; executions a faire une seule fois
    rcall   LCD_init      ; Initialise le LCD
    rcall   LCD_clear     ; clear LCD
    rcall   wire1_init    ; initialise l'interface 1-wire(R)
    rcall   ws2812b_init

    WAIT_MS 200
    PRINTF  LCD
    .db "BIENVENUE"      ; print BIENVENUE sur le LCD

    ; lis le signal IR en loop et saute a la fonction voulue
    main_loop:
        rcall lecture_signal_IR ; lis le signal IR de la telecommande
        WAIT_MS 2          ; delai approximatif equivalent a "cmd="

        ; compare le signal recu et saute au menu voulu

        cpi b0, 0x20      ; Bouton 1
        breq temperature
        cpi b0, 0x33      ;plusieurs code pour la meme fonction a cause de l'irregularite de la telecommande
        breq temperature
        cpi b0, 0x31
        breq temperature
        cpi b0, 0x3f      ; Bouton 'arriere'
        breq distance
        cpi b0,0x3b
        breq distance      ;plusieurs code pour la meme fonction a cause de l'irregularite de la telecommande
        cpi b0,0x2f
        breq distance
        cpi b0,0x2b
        breq distance

        rjmp  main_loop

```

```

temperature:
    rcall mesure_temp    ; lis et ecris la temperature
    rjmp main_loop

distance:
    rcall mesure_dist    ; lis et ecris la distance
    ;compare la valeur de la distance avec la distance de transition
    cpi a0, low(distance_de_transition)
    ldi r16, high(distance_de_transition)
    cpc a1, r16
    brsh distance_limite
    rcall affichage_vert
    rcall ws2812b_freeze_frame
    ; si la limite n'est pas atteinte --> vert
    rjmp main_loop

    ; si la limite est atteinte --> rouge
distance_limite:
    rcall affichage_rouge
    rcall ws2812b_freeze_frame
    rcall LCD_1f
    PRINTF LCD
    .db "    STOP !",0    ;message si proche
    rjmp main_loop

; !! //////////////////////////////////// SOUS-ROUTINES //////////////////////////////////// !!

; Sous-routine message SOS
print_SOS:
    rcall LCD_clear
    PRINTF LCD
    .db "    SOS"
    ret
; ////////////////////////////////////

```

wire1.asm :

```
/*
 * wire1.asm
 *
 */

; === definitions ===
.equ   DQ_port = PORTB
.equ   DQ_pin  = DQ

.equ   DS18B20    = 0x28

.equ   readROM    = 0x33
.equ   matchROM   = 0x55
.equ   skipROM    = 0xcc
.equ   searchROM  = 0xf0
.equ   alarmSearch = 0xec

.equ   writeScratchpad = 0x4e
.equ   readScratchpad  = 0xbe
.equ   copyScratchpad  = 0x48
.equ   convertT        = 0x44
.equ   recallE2        = 0xb8
.equ   readPowerSupply = 0xb4

; === routines ===

.macro WIRE1    ; t0,t1,t2
    sbi DQ_port-1,DQ_pin    ; pull DQ low (DDR=1 output)
    ldi w,(@0+1)/2
    rcall wire1_wait        ; wait low time (t0)
    cbi DQ_port-1,DQ_pin    ; release DQ (DDR=0 input)
    ldi w,(@1+1)/2
    rcall wire1_wait        ; wait high time (t1)
    in  w,DQ_port-2         ; sample line (PINx=PORTx-2)
    bst w,DQ_pin            ; store result in T
    ldi w,(@2+1)/2
    rcall wire1_wait        ; wait separation time (t2)
    ret
.endmacro

wire1_wait:
    dec w                    ; loop time 2usec
    nop
    nop
    nop
    nop
    nop
    brne wire1_wait
    ret
```

```

wire1_init:
    cbi DQ_port, DQ_pin    ; PORT=0 low (for pull-down)
    cbi DQ_port-1,DQ_pin    ; DDR=0 (input hi Z)
    ret

wire1_reset:    WIRE1    480,70,410
wire1_write0:    WIRE1    56,4,1
wire1_write1:    WIRE1    1,59,1
wire1_read1:    WIRE1    1,14,45

wire1_write:
    push    a1
    ldi a1,8
    ror a0

    brcc    PC+3            ; if C=1 then wire1, else wire0
    rcall   wire1_write1
    rjmp    PC+2
    rcall   wire1_write0

    DJNZ    a1,wire1_write+2 ; dec and jump if not zero
    pop a1
    ret

wire1_read:
    push    a1
    ldi a1,8
    ror a0
    rcall   wire1_read1      ; returns result in T
    bld a0,7                ; move T to MSb
    DJNZ    a1,wire1_read+2 ; dec and jump if not zero
    pop a1
    ret

wire1_crc:
    ldi w,0b00011001
    ldi a2,8
crc1:    ror a0
    brcc    PC+2
    eor a1,w
    bst a1,0
    ror a1
    bld a1,7
    DJNZ    a2,crc1
    ret

```

temperature.asm :

```
/*
 * temperature.asm
 */

; === initialisation (reset) ===
temp_reset:
    LDSP    RAMEND        ; load le SP
    rcall   wire1_init     ; initialize l'interface 1-wire(R)
    rcall   lcd_init       ; initialise LCD
    rjmp    mesure_temp

; === main pour mesure de la temperature ===
mesure_temp:
    rcall   wire1_reset    ; send a reset pulse
    CA      wire1_write, skipROM ; skip ROM identification
    CA      wire1_write, convertT ; initiate temp conversion
    ;WAIT_MS 750          ; wait 750 msec on aurait pu contourner 85 deg au debut avec le WAIT mais irrespect du temps reel.

    rcall   LCD_clear
    rcall   lcd_home       ; place le curseur en position initiale
    rcall   wire1_reset    ; send a reset pulse
    CA      wire1_write, skipROM
    CA      wire1_write, readScratchpad
    rcall   wire1_read      ; Lis la temp LSB
    mov     c0,a0
    rcall   wire1_read      ; Lis la temp MSB
    mov     a1,a0
    mov     a0,c0
    PRINTF  LCD

    .db "temperature",FFRAC2+FSIGN,a,4,$42,"C ",CR,0 ; imprime le resultat sur l'ecran LCD
//new code:
    mov     r16, c0        ; LSB dans r16
    mov     r17, a1        ; MSB dans r17

    ldi     r18, low(416) ; code pour le 26
    ldi     r19, high(416); code pour le 26

    cp      r16, r18        ; compare LSB
    cpc     r17, r19        ; compare MSB avec retenue

    brlo    temp_lower
//allume rouge
    rcall   affichage_rouge
    rcall   ws2812b_freeze_frame
    ret
temp_lower:
    //allume bleu
    rcall   affichage_bleu_2
    rcall   ws2812b_freeze_frame
    ret
```

ir_rc5.asm :

```

/*
 * ir_rc5.asm
 */

;livre page 302

reset_IR:
    LDSP      RAMEND      ; load stack pointer SP
    rcall    LCD_init     ; initialize LCD
    rjmp     read_IR      ; jump to main

read_IR:
    CLR2     b1,b0        ; clear 2-byte register
    ldi      b2,14        ; load bit-counter
    WP1 PINE,IR            ; wait if Pin=1
    WAIT_US  (T50/4)      ; wait a quarter period

loop:
    P2C      PINE,IR      ; move Pin to Carry (P2C)
    ROL2     b1,b0        ; roll carry into 2-byte reg
    WAIT_US  (T50-4)      ; wait bit period (- compensation)
    DJNZ     b2,loop      ; Decrement and Jump if Not Zero

    com      b0            ; complement b0
    andi     b0, 0xEF      ; ignorer le toggle bit
    WAIT_MS  1000

    ret

```

distance.asm

```

////////// detecteur de distance //////////

reset_ad:
    LDSP      RAMEND      ; set up le SP
    OUTI     DDRB,0xff     ; configure le portB en output
    rcall    LCD_init     ; initialise le LCD

    OUTI     ADCSR,(1<<ADEN)+6 ; AD Enable, PS=CK/64
    OUTI     ADMUX, 3      ; select channel potentiometre
    rjmp     mesure_dist   ; saut a la fonction de mesure

mesure_dist:
    sbi      ADCSR,ADSC    ; AD start conversion
    WP1      ADCSR,ADSC    ; attend si ADIF=0
    in       a0,ADCL       ; lis le low byte en premier
    in       a1,ADCH       ; lis le high byte ensuite

    rcall    LCD_clear     ; efface ce qui est ecrit sur le LCD
    PRINTF   LCD
    .db      CR,CR,"DISTANCE= ",FDEC2,a," ",0 ;DISTANCE= a
    WAIT_MS  100          ; attend 100 msec

    ret

```

couleurs.asm :

```
/*
 * couleurs.asm
 */

.macro WS2812B_WRITE0
    clr u
    sbi PORTD, 1 ;
    out PORTD, u ;
    nop
    nop
.endm

.macro WS2812B_WRITE1
    sbi PORTD, 1 ;
    nop
    nop
    cbi PORTD, 1 ;
.endm

ws2812b_init:
    OUTI DD RD, 0x02 ; Set PD1 as output ;
    ret

ws2812b_write3:
    ldi w, 8
ws_loop_a0:
    sbrc a0, 7
    rjmp ws_bit1_a0
    WS2812B_WRITE0
    rjmp ws_next_a0
ws_bit1_a0:
    WS2812B_WRITE1

ws_next_a0:
    lsl a0
    dec w
    brne ws_loop_a0

    ldi w, 8
ws_loop_a1:
    sbrc a1, 7
    rjmp ws_bit1_a1
    WS2812B_WRITE0
    rjmp ws_next_a1
ws_bit1_a1:
    WS2812B_WRITE1
ws_next_a1:
    lsl a1
    dec w
    brne ws_loop_a1

    ldi w, 8
ws_loop_a2:
    sbrc a2, 7
    rjmp ws_bit1_a2
    WS2812B_WRITE0
    rjmp ws_next_a2
ws_bit1_a2:
    WS2812B_WRITE1
ws_next_a2:
    lsl a2
    dec w
    brne ws_loop_a2
    ret
```

```
ws2812b_reset:
    cbi PORTD, 1    ;
    ret
```

```
ws2812b_freeze_frame:
    ldi z1, low(0x0400)
    ldi zh, high(0x0400)
    _LDI r0, 64      ; nombre de LEDs (64 x 3 octets RGB)
```

```
freeze_loop:
    ld a0, z+        ; R
    ld a1, z+        ; G
    ld a2, z+        ; B
    cli
    rcall ws2812b_write3 ; envoyer (a0,a1,a2)
    sei
    dec r0
    brne freeze_loop

    rcall ws2812b_reset
    ret
```

```
vert:                                ;pour une LED verte
    ldi a0, 0x05
    st z+, a0
    clr a0
    st z+, a0
    st z+, a0
    ret
```

```
eteint:                                ;pour une LED eteinte
    clr a0
    st z+, a0
    st z+, a0
    st z+, a0
    ret
```

```
orange:                                ;pour une LED orange
    ldi a0,0x08
    st z+,a0
    ldi a0,0x3f
    st z+,a0
    ldi a0,0x00
    st z+,a0
    ret
```

```
rouge:                                ;pour une LED rouge
    clr a0
    st z+,a0
    ldi a0,0x3f
    st z+,a0
    ldi a0,0x00
    st z+,a0
    ret
```

```
bleu:                                  ;pour une LED bleu
    clr a0
    st z+,a0
    st z+,a0
    ldi a0,0x0f
    st z+,a0
    ret
```

; Code pour afficher sur la LED:

```
affichage_rouge:
    ldi z1, low(0x0400)
    ldi zh, high(0x0400)
    ldi b2, 0x40
    rcall rouge
    dec b2
    cpi b2,0x00
    brne PC-3
    ret
```

```
affichage_bleu_2:
    ldi z1, low(0x0400)
    ldi zh, high(0x0400)
    ldi b2, 0x40
    rcall bleu
    dec b2
    cpi b2,0x00
    brne PC-3
    ret
```

```
petit_triangle:
    ldi z1, low(0x0400)
    ldi zh, high(0x0400)

    rcall eteint      ;ligne 1
    rcall eteint
    rcall eteint
    rcall eteint
    rcall eteint
    rcall eteint
    rcall eteint
    rcall eteint
```

```
    rcall eteint      ;ligne 2
    rcall eteint
    rcall eteint
    rcall eteint
    rcall eteint
    rcall eteint
    rcall eteint
```

```
    rcall eteint      ;ligne 3
    rcall eteint
    rcall eteint
    rcall orange
    rcall eteint
    rcall eteint
    rcall eteint
```

```
    rcall eteint      ;ligne 4
    rcall eteint
    rcall orange
    rcall eteint
    rcall orange
    rcall eteint
    rcall eteint
```

```
    rcall eteint      ;ligne 5
    rcall orange
    rcall orange
    rcall orange
    rcall orange
    rcall eteint
    rcall eteint
```

```

rcall eteint      ;ligne 6
rcall eteint
rcall eteint
rcall eteint
rcall eteint
rcall eteint
rcall eteint
rcall eteint

rcall eteint      ;ligne 7
rcall eteint
rcall eteint
rcall eteint
rcall eteint
rcall eteint
rcall eteint
rcall eteint

rcall eteint      ;ligne 8
rcall eteint
rcall eteint
rcall eteint
rcall eteint
rcall eteint
rcall eteint
rcall eteint

ret

```

```

grand_triangle:
    ldi z1, low(0x0400)
    ldi zh, high(0x0400)

    rcall eteint      ;ligne 1
    rcall eteint
    rcall eteint
    rcall orange
    rcall orange
    rcall eteint
    rcall eteint
    rcall eteint

    rcall eteint      ;ligne 2
    rcall eteint
    rcall eteint
    rcall orange
    rcall orange
    rcall eteint
    rcall eteint
    rcall eteint

    rcall eteint      ;ligne 3
    rcall eteint
    rcall orange
    rcall orange
    rcall orange
    rcall orange
    rcall eteint
    rcall eteint

```

```
rcall eteint          ;ligne 4
rcall eteint
rcall orange
rcall orange
rcall orange
rcall orange
rcall eteint
rcall eteint
```

```
rcall eteint          ;ligne 5
rcall orange
rcall orange
rcall orange
rcall orange
rcall orange
rcall orange
rcall orange
rcall eteint
```

```
rcall eteint          ;ligne 6
rcall orange
rcall orange
rcall orange
rcall orange
rcall orange
rcall orange
rcall orange
rcall eteint
```

```
rcall orange          ;ligne 7
rcall orange
rcall orange
rcall orange
rcall orange
rcall orange
rcall orange
rcall orange
```

```
rcall orange          ;ligne 8
rcall orange
rcall orange
rcall orange
rcall orange
rcall orange
rcall orange
rcall orange
```

```
ret
```

```
affichage_orange:
    ldi z1, low(0x0400)
    ldi zh, high(0x0400)
    ldi b2, 0x40
    rcall orange
    dec b2
    cpi b2,0x00
    brne PC-3
    ret
```

```
affichage_vert:
    ldi z1, low(0x0400)
    ldi zh, high(0x0400)
    ldi b2, 0x40
    rcall vert
    dec b2
    cpi b2,0x00
    brne PC-3
    ret
```

```
affichage_eteint:
    ldi z1, low(0x0400)
    ldi zh, high(0x0400)
    ldi b2, 0x40
    rcall eteint
    dec b2
    cpi b2,0x00
    brne PC-3
    ret
```

